

Bloque 3: Relaciones entre entidades

Objetivo del bloque

Entender cómo se modelan relaciones reales entre objetos y cómo se traducen a relaciones en base de datos. Este es el punto más importante del ORM, ya que aquí aparecen la mayoría de problemas reales.

1. El problema de las relaciones

En aplicaciones reales, las entidades no están aisladas.

Ejemplo típico:

- Un usuario tiene pedidos
- Un pedido tiene productos

En Java esto se representa con referencias entre objetos.

En base de datos se representa con claves foráneas.

Aquí es donde ORM tiene que traducir correctamente entre ambos mundos.

2. Tipos de relaciones en ORM

JPA define cuatro tipos principales de relaciones:

- OneToOne (uno a uno)
 - OneToMany (uno a muchos)
 - ManyToOne (muchos a uno)
 - ManyToMany (muchos a muchos)
-

3. Relación ManyToOne y OneToMany (la más importante)

Ejemplo conceptual

Un usuario puede tener muchos pedidos.

En base de datos:

Tabla usuarios

- id
- nombre

Tabla pedidos

- id
- fecha
- usuario_id (clave foránea)

Representación en Java

Lado "muchos" (Pedido)

@Entity

```
public class Pedido {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private int id;
```

```
    private String fecha;
```

```
    @ManyToOne
```

```
    private Usuario usuario;
```

```
}
```

Lado "uno" (Usuario)

@Entity

```
public class Usuario {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private int id;
```

```
    private String nombre;
```

```
    @OneToMany(mappedBy = "usuario")
```

```
    private List<Pedido> pedidos;
```

```
}
```

Idea clave

- La clave foránea está en la tabla pedidos

- Por eso el lado importante es el ManyToOne
 - Ese lado es el que “manda” en la relación
-

4. mappedBy (concepto crítico)

`@OneToMany(mappedBy = "usuario")`

Significa:

"La relación ya está gestionada por el atributo usuario en la entidad Pedido"

Sin mappedBy:

- JPA crea una tabla intermedia innecesaria

Con mappedBy:

- Se usa la clave foránea real
-

5. Relación OneToOne

Ejemplo

Un usuario tiene un perfil.

Base de datos:

Tabla usuarios

Tabla perfiles (con usuario_id)

En Java

```
@Entity
public class Usuario {

    @Id
    private int id;

    @OneToOne
    private Perfil perfil;
}
```

@Entity

```
public class Perfil {  
  
    @Id  
    private int id;  
  
    @OneToOne  
    private Usuario usuario;  
}
```

Idea clave

Se usa cuando:

- Una entidad está asociada a exactamente otra
-

6. Relación ManyToMany

Ejemplo

Un pedido tiene muchos productos

Un producto puede estar en muchos pedidos

En base de datos

Se necesita una tabla intermedia:

pedido_producto

- pedido_id
 - producto_id
-

En Java

```
@Entity  
public class Pedido {  
  
    @Id  
    private int id;  
  
    @ManyToMany  
    private List<Producto> productos;  
}
```

```
@Entity
public class Producto {

    @Id
    private int id;

    @ManyToMany(mappedBy = "productos")
    private List<Pedido> pedidos;
}
```

Idea clave

ORM crea automáticamente la tabla intermedia si no se define manualmente.

7. FetchType (carga de datos)

Indica cuándo se cargan los datos relacionados.

Tipos:

EAGER (inmediato)

```
@OneToMany(fetch = FetchType.EAGER)
```

- Carga los datos automáticamente
 - Puede afectar al rendimiento
-

LAZY (diferido)

```
@ManyToOne(fetch = FetchType.LAZY)
```

- No carga los datos hasta que se accede a ellos
 - Más eficiente en general
-

Idea clave

- LAZY es la opción recomendada en la mayoría de casos
 - EAGER puede generar consultas innecesarias
-

8. Cascadas (CascadeType)

Las cascadas definen qué ocurre con entidades relacionadas cuando se realiza una operación.

Ejemplo:

```
@OneToMany(mappedBy = "usuario", cascade = CascadeType.ALL)
```

Significa:

- Si guardo un usuario → guarda sus pedidos
 - Si borro un usuario → borra sus pedidos
-

Tipos comunes:

- PERSIST
 - REMOVE
 - MERGE
 - ALL
-

9. Ejemplo completo (modelo real)

Usuario

```
@Entity
public class Usuario {

    @Id
    @GeneratedValue
    private int id;

    private String nombre;

    @OneToMany(mappedBy = "usuario", cascade = CascadeType.ALL)
    private List<Pedido> pedidos;
}
```

Pedido

```
@Entity
public class Pedido {

    @Id
```

```
@GeneratedValue
private int id;

private String fecha;

@ManyToOne
private Usuario usuario;

@ManyToMany
private List<Producto> productos;
}
```

Producto

```
@Entity
public class Producto {

    @Id
    @GeneratedValue
    private int id;

    private String nombre;

    private double precio;

    @ManyToMany(mappedBy = "productos")
    private List<Pedido> pedidos;
}
```

10. Qué está ocurriendo realmente

Cuando se definen relaciones:

- JPA crea claves foráneas
- Puede crear tablas intermedias
- Genera joins automáticamente en consultas
- Sincroniza objetos relacionados

Ejemplo conceptual:

```
pedido.getUsuario().getNombre();
```

Puede generar internamente:

```
SELECT u.nombre
FROM pedidos p
```

JOIN usuarios u ON p.usuario_id = u.id

11. Problemas típicos en este bloque

Este bloque es donde más errores aparecen:

- No entender quién es el lado dueño de la relación
 - Uso incorrecto de mappedBy
 - Problemas con LAZY (datos no cargados)
 - Uso excesivo de EAGER (problemas de rendimiento)
 - Cascadas mal configuradas
-

12. Idea final del bloque

Las relaciones son el núcleo del ORM.

No basta con mapear clases simples:

- Hay que entender cómo se conectan entre ellas
- Cómo se representan en base de datos
- Cómo JPA gestiona esas relaciones internamente

Dominar este bloque es lo que realmente diferencia a alguien que “usa JPA” de alguien que lo entiende.

13. Ejercicio de ejemplo

Ejemplo: Usuario → Pedidos (OneToMany /ManyToOne)

Clase Usuario

```
import jakarta.persistence.*;  
import java.util.ArrayList;  
import java.util.List;
```

```
@Entity
```

```
public class Usuario {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private int id;
```

```
    private String nombre;
```

```
    @OneToMany(mappedBy = "usuario", cascade = CascadeType.ALL)
```



```
private List<Pedido> pedidos = new ArrayList<>();

public void addPedido(Pedido p) {
    pedidos.add(p);
    p.setUsuario(this);
}

// getters y setters
}
```

Clase Pedido

```
import jakarta.persistence.*;

@Entity
public class Pedido {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String descripcion;

    @ManyToOne
    private Usuario usuario;

    // getters y setters
}
```

Clase Main

```
import jakarta.persistence.*;

public class Main {

    public static void main(String[] args) {

        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("miUnidad");

        EntityManager em = emf.createEntityManager();

        em.getTransaction().begin();

        Usuario u = new Usuario();
        u.setNombre("Ana");

        Pedido p1 = new Pedido();
```

```
        p1.setDescripcion("Pedido 1");

        Pedido p2 = new Pedido();
        p2.setDescripcion("Pedido 2");

        u.addPedido(p1);
        u.addPedido(p2);

        em.persist(u);

        em.getTransaction().commit();

        em.close();
        emf.close();
    }
}
```

EJERCICIOS

1. Usuario → Pedidos (OneToMany)

Crear usuario con lista de pedidos y persistir todo.

2. Pedido → Usuario (ManyToOne)

Crear pedido asociado a usuario existente.

3. Pedido → Productos (ManyToMany)

Agregar productos a pedidos.

4. Crear pedido completo con relaciones

Usuario + Pedido + Productos en una sola transacción.

5. Crea una relación Usuario → Perfil (OneToOne). Guarda un usuario con su perfil asociado.

6. Crea una relación Usuario → Pedido (OneToMany / ManyToOne). Añade un método addPedido en Usuario y guarda un usuario con dos pedidos.

7. Crea solo la entidad Pedido con ManyToOne hacia Usuario. Asocia un pedido a un usuario ya existente y guárdalo.

8. Crea una relación Usuario → Perfil (OneToOne). Guarda un usuario con su perfil asociado.

9. Crea una relación Pedido → Producto (ManyToMany). Añade dos productos a un pedido y persiste todo.